Chapitre 2 : Virtualisation classique

Module : Cloud Computing, Virtualisation & DevOps — Master Sciences et Techniques

Pr. A. ABASSI

Département de Physique Appliquée
Laboratoire LAVETE Lab – FST Settat

Faculté des Sciences et Techniques de Settat
Université Hassan 1er

October 2, 2026

Sommaire

1. Introduction
2. Partie I — Qu'est-ce qu'une machine virtuelle ?
3. Partie II — Les deux types d'hyperviseurs
4. Partie III — Virtualisation complète, virtualisation matérielle
5. Partie IV — VirtualBox en pratique
6. Partie VI — Disques virtuels
7. Partie VII — Snapshots
8. Partie VIII — Clonage de VM
9. Partie IX — Gestion des ressources partagées
10. Partie X — Migration de VM
11. Partie XI — Virtualisation vs conteneurisation
12. Étude de cas de synthèse
13. Synthèse
14. Mini-quiz
15. TP guidé
16. Conclusion

1. Introduction

Le problème du ń un serveur, une application ż

Pratique historique

Avant la virtualisation, chaque application critique était installée sur **sa propre machine physique**, pour éviter qu'un problème sur une application n'affecte les autres.

Conséquence : un gaspillage massif

- Un serveur de messagerie utilisant 10% de son CPU en moyenne **immobilise 100%** de la machine.
- Multiplier les machines physiques multiplie les coûts : achat, électricité, climatisation, espace physique, maintenance.
- Ajouter un nouveau service signifie acheter un **nouveau serveur**, avec un délai de plusieurs semaines.

L'idée de la virtualisation

Faire tourner **plusieurs machines logiques indépendantes** sur une **seule machine physique**, chacune croyant disposer de son propre matériel dédié.

Ce que nous allons construire, pas à pas

1. Comprendre **ce qu'est** une machine virtuelle et un hyperviseur
2. Distinguer les **deux types** d'hyperviseurs
3. Comprendre l'**architecture** d'un système virtualisé
4. Manipuler **VirtualBox** : création, configuration, démarrage d'une VM
5. Comprendre les **disques virtuels** et leurs formats
6. Utiliser les **snapshots** pour figer un état
7. **Cloner** une VM pour la dupliquer rapidement
8. Gérer les **ressources** (CPU, RAM) partagées entre VM
9. Situer la virtualisation classique par rapport à la **conteneurisation** (aperçu du chapitre suivant)

2. Partie I — Qu'est-ce qu'une machine virtuelle ?

Définition

Machine virtuelle (VM)

Une **machine virtuelle** est une émulation logicielle d'un ordinateur complet : elle possède son propre CPU virtuel, sa propre mémoire, son propre disque et ses propres interfaces réseau **virtuels**, et exécute son **propre système d'exploitation**, totalement indépendant de celui de la machine physique qui l'héberge.

Point clé

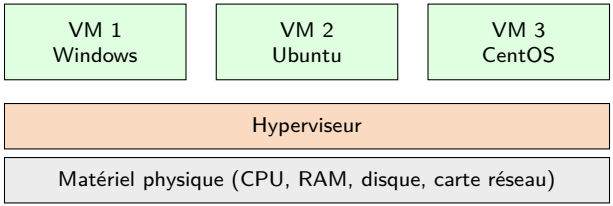
Du point de vue du système d'exploitation installé **dans** la VM, tout se comporte comme s'il s'exécutait sur du matériel physique réel — il ń ne sait pas ż qu'il est virtualisé (sauf pilotes spécifiques d'optimisation).

Machine physique (hôte)	**Machine virtuelle (invité)**
Matériel réel	Matériel émulé/virtualisé
Un seul système d'exploitation hôte	Un ou plusieurs systèmes d'exploitation invités

Le rôle central de l'hyperviseur

Hyperviseur (*Virtual Machine Monitor*)

Le logiciel qui crée, exécute et gère les machines virtuelles. Il alloue les ressources physiques réelles (CPU, RAM, disque) entre les différentes VM, et s'assure qu'elles restent **isolées** les unes des autres.



Chaque VM peut exécuter un système d'exploitation **totalement différent** des autres — l'hyperviseur ne se soucie que de leur fournir des ressources virtuelles.

Mini-exercice — Vocabulaire

Associer chaque terme à sa définition :

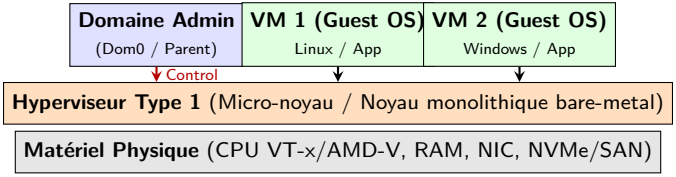
Terme	Définition
1. Hôte	a. Le logiciel qui gère les VM
2. Invité	b. La machine physique réelle
3. Hyperviseur	c. Le système d'exploitation exécuté dans une VM

3. Partie II — Les deux types d'hyperviseurs

Hyperviseur de Type 1 (*Bare-Metal*) : Architecture

Définition & Principe

S'installe **directement sur le matériel physique** (*bare-metal*), sans système d'exploitation hôte. L'hyperviseur prend le contrôle total du CPU, de la RAM et du bus d'E/S dès le démarrage de la machine.



Solutions industrielles majeures

VMware ESXi (vSphere), **KVM** (Kernel-based VM / Proxmox / Red Hat), **Microsoft Hyper-V Core**, **XenServer / Citrix Hypervisor**.

Trois notions à ne pas confondre

Dans le monde de la virtualisation

Les termes **hyperviseur**, **technologie de virtualisation** et **plateforme de gestion** désignent des éléments différents, même s'ils sont souvent utilisés ensemble.

Notion	Rôle	Exemple
Hyperviseur	Exécute les VM et contrôle l'accès aux ressources physiques.	VMware ESXi, Xen, Hyper-V
Technologie de virtualisation	Composant ou mécanisme permettant au système d'exécuter des VM.	KVM
Plateforme de gestion	Fournit des outils pour créer, configurer, surveiller et administrer les VM.	Proxmox VE, VMware vSphere

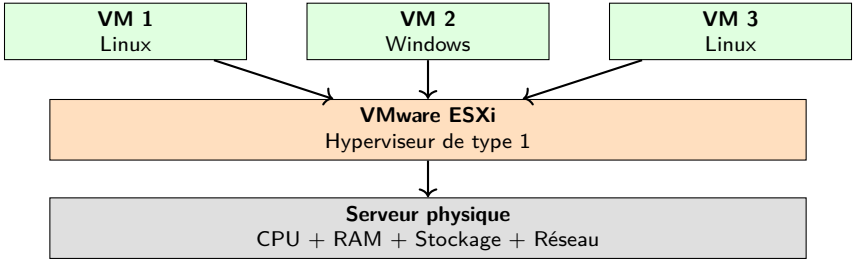
À retenir

Un hyperviseur fait fonctionner les VM.
Une plateforme de gestion facilite leur administration.

Exemple 1 : VMware ESXi et vSphere

VMware ESXi

ESXi est un **hyperviseur de type 1**. Il s'installe directement sur le serveur physique et permet d'exécuter plusieurs machines virtuelles.



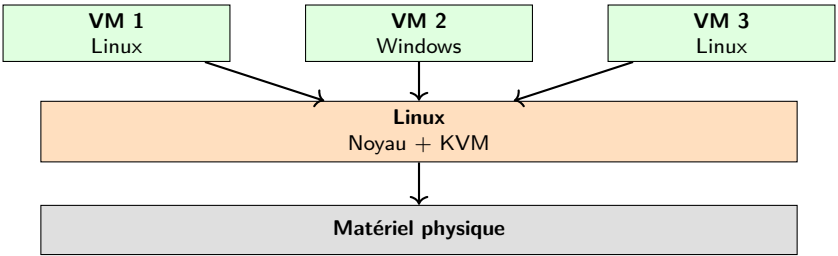
Et vSphere ?

vSphere désigne une **plateforme/solution plus large** permettant notamment de gérer des environnements utilisant ESXi : création, administration, supervision et gestion des ressources.

Exemple 2 : KVM et Proxmox VE

KVM

KVM (Kernel-based Virtual Machine) est une technologie de virtualisation intégrée au **noyau Linux**. Elle permet à Linux d'utiliser les fonctions de virtualisation matérielle du processeur pour exécuter des machines virtuelles.



Et Proxmox VE ?

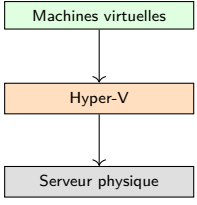
Proxmox VE est une **plateforme de virtualisation et de gestion**. Elle utilise notamment **KVM** pour les machines virtuelles et fournit une interface Web pour les administrer.

Exemple 3 : Microsoft Hyper-V et Xen

Microsoft Hyper-V

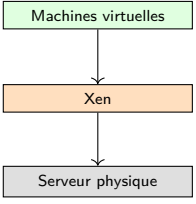
Hyper-V est la technologie de virtualisation de Microsoft permettant d'exécuter plusieurs VM sur un serveur physique.

Elle est notamment disponible dans **Windows Server**.



Xen

Xen est un **hyperviseur de type 1**. Il permet d'exécuter plusieurs systèmes d'exploitation invités sur le même matériel physique.



Exemples de solutions

Hyper-V : écosystème Microsoft

Xen : hyperviseur open source utilisé par différentes plateformes de virtualisation

Synthèse : qui fait quoi ?

Solution	Catégorie	Rôle
VMware ESXi	Hyperviseur Type 1	Exécute directement les VM sur le serveur physique.
KVM	Technologie de virtualisation Linux	Permet à Linux d'exécuter des machines virtuelles.
Proxmox VE	Plateforme de gestion	Permet de créer, administrer et superviser des VM et des conteneurs. Utilise notamment KVM.
Microsoft Hyper-V	Technologie / hyperviseur Microsoft	Permet d'exécuter des VM dans l'écosystème Microsoft.
Xen	Hyperviseur Type 1	Permet l'exécution de plusieurs systèmes invités sur un même serveur.
VMware vSphere	Plateforme / suite de virtualisation	Fournit un environnement plus large pour administrer des infrastructures utilisant notamment ESXi.

Idée essentielle

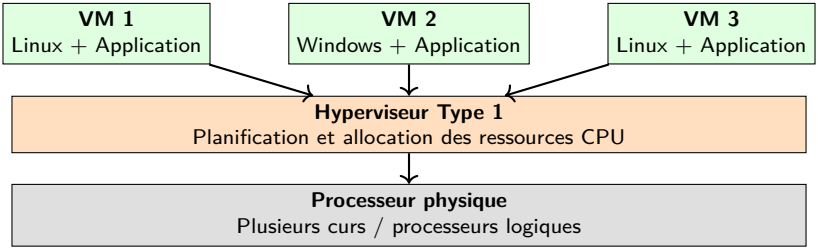
Hyperviseur = faire fonctionner les VM

Technologie = mécanisme permettant la virtualisation

Hyperviseur Type 1 : comment le CPU exécute les VM

Principe fondamental

L'hyperviseur de type 1 **partage les ressources physiques** entre les différentes machines virtuelles. Il attribue notamment des **CPU logiques / curs** aux VM selon leur charge et les ressources disponibles.



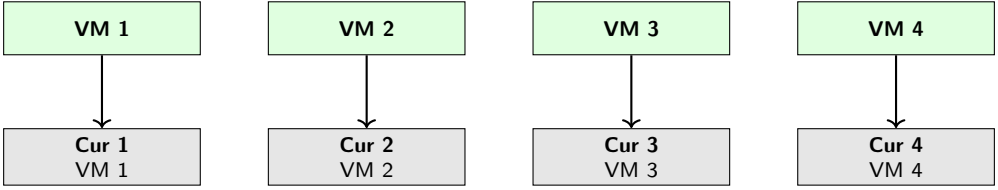
Point important

La virtualisation ne signifie pas que les VM sont exécutées nécessairement l'une après l'autre.
Avec plusieurs curs, plusieurs VM peuvent être exécutées **simultanément**.

Hyperviseur Type 1 : exécution parallèle des VM

Cas d'un processeur multicur

Si le serveur possède plusieurs curs, l'hyperviseur peut affecter différents curs à différentes VM. Les instructions des VM peuvent alors être exécutées **en parallèle**.



Exemple

Un serveur possède **4 curs** et héberge 4 VM actives. Si les ressources sont disponibles, chaque VM peut utiliser un cur différent : les quatre VM peuvent donc progresser **simultanément**.

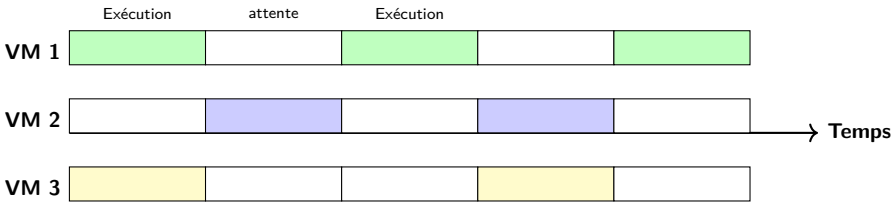
À retenir

Plusieurs curs physiques ⇒ possibilité d'exécution réellement parallèle.

Hyperviseur Type 1 : partage du temps CPU

Que se passe-t-il lorsque les VM sont plus nombreuses que les curs ?

Lorsque plusieurs VM demandent simultanément plus de CPU que les ressources physiques disponibles, l'hyperviseur utilise un **ordonnancement** (*scheduling*) pour partager le temps processeur.



Exemple

Avec **1 seul cur** et plusieurs VM, le processeur ne peut pas exécuter leurs instructions au même instant. L'hyperviseur alterne rapidement entre les VM.

VM 1 $\rightarrow$ **VM 2** $\rightarrow$ **VM 3** $\rightarrow$ **VM 1** $\rightarrow$ **...**

Virtualisation : parallèle, séquentiel ou synchrone ?

Trois notions à ne pas confondre

Le fonctionnement des VM fait intervenir plusieurs concepts différents. Il faut distinguer **parallélisme**, **exécution séquentielle** et **synchronisation**.

Notion	**Signification**	**Dans une virtualisation Type 1**
Parallèle	Plusieurs unités de calcul travaillent simultanément.	Possible avec plusieurs curs / CPU.
Séquentiel	Les opérations sont exécutées l'une après l'autre.	Peut apparaître lorsqu'un seul cur doit servir plusieurs VM.
Synchrone	Des opérations sont coordonnées ou doivent attendre certains événements.	Concerne la synchronisation, et non directement le nombre de VM exécutées simultanément.

Attention

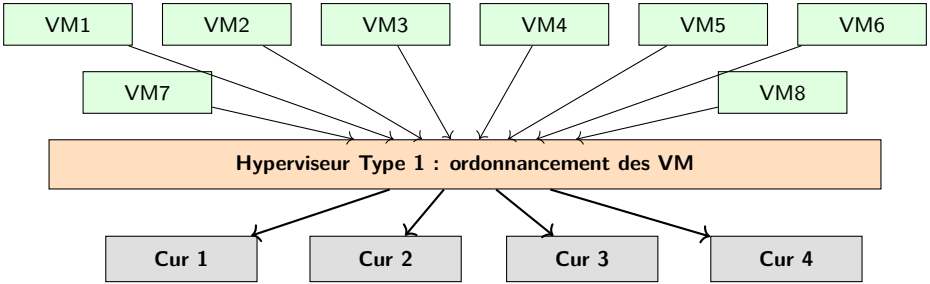
ń Parallèle ż et ń synchrone ż ne sont pas des notions opposées.

Une exécution peut être **parallèle tout en utilisant des mécanismes de synchronisation**.

Exemple : serveur Type 1 avec 4 curs et 8 VM

Situation

Un serveur physique possède **4 curs CPU** et héberge **8 machines virtuelles**. Les 8 VM peuvent être actives simultanément, mais seulement 4 curs sont disponibles à un instant donné.



Ce qu'il faut comprendre

- Les **8 VM peuvent être actives** en même temps.
- Les **4 curs** permettent au maximum 4 exécutions CPU réellement simultanées à un instant donné.
- Les autres VM attendent ou partagent le temps CPU.

Hyperviseur de Type 1 : Mécanismes Internes

Comment l'Hyperviseur Type 1 gère-t-il les ressources ?

- **Assistance matérielle CPU (Ring -1) :** Utilise les instructions processeur dédiées (**Intel VT-x** ou **AMD-V**). Les systèmes invités (*Guest OS*) s'exécutent en Ring 0 virtuel sans altérer la sécurité de la machine hôte.
- **Gestion de la mémoire physique (SLAT / EPT) :** L'hyperviseur convertit directement la mémoire virtuelle de la VM en mémoire physique réelle via des tables de pages matérielles (*Extended Page Tables*).
- **Gestion des Entrées/Sorties (I/O) :**
  - *Pas d'émulation lourde :* Utilisation de pilotes para-virtuels (`virtio`) ou d'accès direct au matériel (**SR-IOV / PCIe Passthrough**).

Le rôle de la VM d'Administration (Dom0 / Parent Partition)

Un hyperviseur Bare-Metal minimaliste n'a souvent **pas de GUI ni de pile réseau de gestion complète** :

- Une VM privilégiée (*Dom0* sous Xen, *Parent Partition* sous Hyper-V) est chargée d'exécuter l'interface de gestion, les agents de sauvegarde et la pile réseau de contrôle.

Hyperviseur de Type 1 : Avantages & Usages Cloud

Avantages clés

- **Performances Quasi-Natives :** Overheads très faibles ($< 2\%$ à 5%).
- **Isolation Stricte :** Aucune faille venant d'un OS hôte classique.
- **Densité Élevée :** Des dizaines/centaines de VMs par nud physique.
- **Haute Disponibilité :** Support natif du *vMotion* / Live Migration.

Contraintes & Limites

- **Compatibilité matérielle :** Nécessite du matériel serveur certifié (cartes NIC, RAID).
- **Gestion à distance :** Administration via API / Console dédiée (ex: vSphere Client).

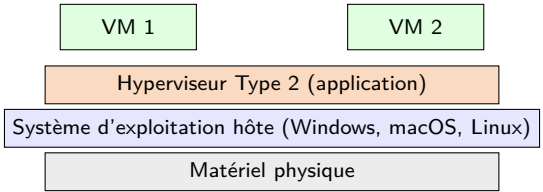
Usage Typique en Cloud

Socle exclusif des **Datacenters**, des **Cloud Publics** (AWS EC2 Nitro, Azure Hyper-V, GCP KVM) et des **Cloud Privés**.

Hyperviseur de Type 2 (*hosted*)

Définition

S'installe **comme une application** par-dessus un système d'exploitation hôte déjà existant.



Exemples

VirtualBox, VMware Workstation/Fusion, Parallels Desktop.

Usage typique

Poste personnel, pédagogie, dév./tests — installation simple, performances légèrement inférieures au Type 1.

Tableau comparatif Type 1 vs Type 2

Critère	Type 1 (bare-metal)	Type 2 (hosted)
Installation	Directement sur le matériel	Par-dessus un OS existant
Performances	Proches du natif	Légèrement inférieures
Complexité d'installation	Plus élevée	Simple
Usage typique	Datacenter, production, Cloud	Poste personnel, pédagogie, tests
Exemples	ESXi, Hyper-V, KVM, Xen	VirtualBox, VMware Workstation

Dans ce cours

Nous utiliserons **VirtualBox** (Type 2), pour sa simplicité d'installation sur un poste personnel — les concepts restent identiques à ceux utilisés en production avec un hyperviseur de Type 1.

Mini-exercice — Type 1 ou Type 2 ?

Pour chaque scénario, indiquer s'il s'agit plutôt d'un usage Type 1 ou Type 2, et justifier :

1. Un fournisseur Cloud héberge des milliers de VM clientes sur ses serveurs de datacenter.
2. Un étudiant installe VirtualBox sur son PC portable Windows pour suivre ce cours.
3. Une entreprise virtualise son parc de serveurs de production pour optimiser leurs coûts.

4. Partie III — Virtualisation complète, virtualisation matérielle

Comment l'hyperviseur isole-t-il chaque VM ?

- **CPU virtuel** : l'hyperviseur planifie l'exécution des instructions de chaque VM sur les curs CPU physiques réels, en s'appuyant souvent sur des extensions matérielles dédiées (Intel VT-x, AMD-V) pour de meilleures performances.
- **Mémoire virtuelle** : chaque VM reçoit une plage de RAM qui lui semble dédiée, alors qu'elle est en réalité gérée et isolée par l'hyperviseur au sein de la RAM physique totale.
- **Disque virtuel** : un simple **fichier** sur le disque physique de l'hôte, présenté à la VM comme un disque dur complet (voir Partie VI).
- **Réseau virtuel** : une carte réseau logicielle, reliée aux modes vus au Chapitre ń Réseaux, sécurité et IaC ż (NAT, Host-only, Bridged...).

À retenir

Toute l'astuce de la virtualisation consiste à faire correspondre des ressources **virtuelles**, présentées à chaque VM comme si elles étaient dédiées, à des ressources **physiques réelles**, en réalité partagées et arbitrées par l'hyperviseur.

5. Partie IV — VirtualBox en pratique

Vue d'ensemble de VirtualBox

VirtualBox est un hyperviseur de Type 2, gratuit et open-source, développé initialement par Sun Microsystems puis maintenu par Oracle.

Composants principaux de l'interface

- Le **Gestionnaire VirtualBox** : liste et administre toutes les VM créées
- L'**assistant de création** : guide pas à pas la création d'une nouvelle VM
- Les **paramètres de VM** : CPU, RAM, disque, réseau, affichage
- La **fenêtre d'exécution** : affiche l'écran de la VM en cours de fonctionnement

Créer une VM, étape par étape

1. **Nom et système** : choisir un nom, le type (Linux, Windows...) et la version (Ubuntu, Debian...) — VirtualBox adapte certains paramètres par défaut selon ce choix.
2. **Mémoire vive (RAM)** : allouer une quantité de RAM à la VM, prélevée sur la RAM physique de l'hôte pendant l'exécution.
3. **Disque dur virtuel** : créer un nouveau disque virtuel ou réutiliser un disque existant (voir Partie VI).
4. **Support d'installation** : associer une image **ISO** du système d'exploitation à installer (ex. `ubuntu-22.04-desktop-amd64.iso`), montée comme un lecteur optique virtuel.
5. **Démarrage** : lancer la VM, qui démarre alors sur l'ISO monté, comme le ferait un ordinateur physique avec un DVD d'installation.

Attention

L'ISO n'est **montée** que temporairement pour l'installation — une fois le système installé sur le disque virtuel, on retire l'ISO pour que la VM démarre directement dessus.

Dimensionner les ressources d'une VM

Ressource	Sous-dimensionner	Sur-dimensionner
RAM	VM lente, blocages, échecs	RAM indisponible pour l'hôte et les autres VM
CPU (curs)	Traitement lent	Contention CPU si plusieurs VM actives
Disque	Manque d'espace en cours d'usage	Gaspillage d'espace disque physique

Règle de bon sens

Ne jamais allouer **plus** de RAM ou de curs CPU virtuels que ce que la machine hôte possède réellement de disponible — sous peine de ralentir **à la fois** l'hôte et la VM.

Question de vérification

Une machine hôte possède 16 Go de RAM et 8 curs CPU. Est-il raisonnable de créer 3 VM avec 8 Go de RAM chacune, actives simultanément ? Justifier.

6. Partie VI — Disques virtuels

Disque virtuel : un simple fichier

Principe

Un disque virtuel est, physiquement, **un fichier** stocké sur le disque de la machine hôte, mais présenté **à l'intérieur** de la VM comme un disque dur complet, avec sa propre table de partitions.

Format	Hyperviseur associé	Extension
VDI	VirtualBox (natif)	`.vdi`
VMDK	VMware (mais lisible par VirtualBox)	`.vmdk`
VHD/VHDX	Microsoft Hyper-V	`.vhd` / `.vhdx`

À retenir

VirtualBox peut lire plusieurs formats (VDI, VMDK, VHD), ce qui facilite la migration de VM entre différents hyperviseurs.

Disque à taille fixe vs dynamiquement alloué

Critère	Taille fixe	Dynamiquement alloué
Espace disque réservé	Immédiatement, en totalité	Progressivement, selon l'usage réel
Performances	Légèrement meilleures	Légèrement inférieures
Flexibilité	Faible	Élevée
Usage typique	Production, performance critique	Pédagogie, tests, usage courant

Exemple

Un disque virtuel ń dynamiquement alloué ż de 50 Go configuré n'occupera, au départ, que quelques centaines de Mo réels sur le disque de l'hôte — il grandit au fur et à mesure que la VM y écrit des données, jusqu'à la limite de 50 Go.

Attention

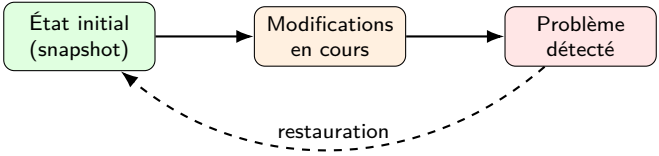
Un disque dynamique peut donner l'illusion d'un espace disque hôte disponible supérieur à la réalité si plusieurs VM sont configurées avec des tailles maximales dynamiques cumulées supérieures à l'espace physique réel.

7. Partie VII — Snapshots

Le principe du snapshot

Snapshot (instantané)

Une **capture de l'état complet** d'une VM à un instant donné (disque, mémoire, configuration), permettant d'y **revenir** ultérieurement, comme une machine à remonter le temps.



Cas d'usage typiques

- Avant une mise à jour risquée du système : ń filet de sécurité ż
- Avant de tester une manipulation potentiellement destructrice en TP
- Créer plusieurs points de reprise pendant un développement

Les limites des snapshots

Un snapshot n'est pas une sauvegarde

- Un snapshot est stocké **sur le même disque physique** que la VM : si le disque de l'hôte est corrompu ou détruit, le snapshot est perdu **avec** la VM.
- Accumuler des snapshots **dégrade les performances** et consomme un espace disque croissant.
- Un snapshot capture un état technique, pas nécessairement un état **cohérent** du point de vue applicatif (ex. transaction de base de données en cours).

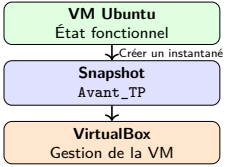
À retenir

Pour une véritable protection contre la perte de données, il faut une **sauvegarde** sur un support **physiquement distinct** — le snapshot est un outil de confort, pas une stratégie de sauvegarde.

Exemple concret : snapshot avec VirtualBox

Situation

On dispose d'une machine virtuelle **Ubuntu** créée avec **VirtualBox**. Avant une manipulation importante, on crée un snapshot appelé `Avant_TP`.



Dans VirtualBox

1. Sélectionner la VM **Ubuntu**.
2. Ouvrir la rubrique **Snapshots / Instantanés**.
3. Cliquer sur **Take / Prendre un snapshot**.
4. Donner le nom : `Avant_TP`.
5. Valider la création du snapshot.

Objectif

Le snapshot constitue un **point de retour** avant d'effectuer une manipulation risquée.

VirtualBox : après la création du snapshot

Étape 2 : modification de la machine virtuelle

Après avoir créé le snapshot `Avant_TP`, on démarre Ubuntu et on réalise différentes manipulations.



Exemple de manipulation

On peut par exemple :

- installer un logiciel ;
- modifier une configuration réseau ;
- installer des paquets Linux ;
- supprimer accidentellement un fichier ;
- modifier plusieurs paramètres du système.

Problème

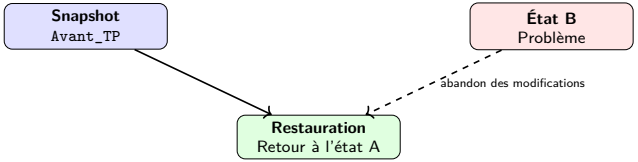
Supposons que les modifications rendent finalement le système instable ou inutilisable.

Comment revenir rapidement à l'état précédent ?

VirtualBox : restaurer le snapshot

Étape 3 : retour à l'état précédent

Dans VirtualBox, on sélectionne le snapshot `Avant_TP` puis on choisit l'option **Restore / Restaurer**.



Résultat

Après restauration, la VM retrouve essentiellement l'état enregistré au moment du snapshot :

- fichiers et logiciels présents à cet instant ;
- configuration du système ;
- état des disques virtuels ;
- selon le type de snapshot et les options utilisées, certains éléments d'exécution peuvent également être restaurés.

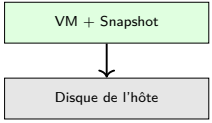
Attention

Les modifications effectuées **après le snapshot** peuvent être perdues lors de la restauration.

Exemple VirtualBox : snapshot $\neq$ sauvegarde

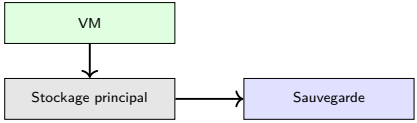
Snapshot

- Point de retour rapide.
- Utile avant une manipulation risquée.
- Très pratique pour les TP.
- Généralement conservé avec les fichiers de la VM.
- Peut dépendre du disque de stockage de la VM.



Sauvegarde

- Protection contre la perte de données.
- Copie vers un autre emplacement.
- Peut être conservée sur un autre disque.
- Peut être restaurée même après une panne du stockage principal.
- Fait partie d'une véritable stratégie de sauvegarde.



Scénario pédagogique

Avant le TP : Snapshot → Pendant le TP : modifications → Erreur : Restore

Pour protéger réellement les données : **Backup sur un support distinct**.

8. Partie VIII — Clonage de VM

Cloner une VM

Le clonage permet de dupliquer rapidement une VM existante, pour éviter de refaire toute une installation manuelle.

Type	Fonctionnement	Indépendance
Clone complet	Copie **intégrale** du disque	Totale, même si l'originale est supprimée
Clone lié	Ne stocke que les **différences**	Dépend de la VM d'origine

Cas d'usage

Un enseignant prépare **une** VM de référence pour un TP, puis crée 20 **clones complets**, un par étudiant, chacun totalement indépendant.

Attention

Cloner une VM **ne change pas** automatiquement son adresse réseau ni son identifiant système (SID sous Windows) : une réinitialisation peut être nécessaire pour éviter des conflits.

9. Partie IX — Gestion des ressources partagées

Le surengagement (*overcommitment*)

Définition

Allouer, sur le papier, **plus** de ressources virtuelles (RAM, CPU) à l'ensemble des VM que ce dont dispose réellement la machine physique — pari que **toutes** les VM n'utiliseront jamais leur maximum **simultanément**.

Exemple

Un serveur physique avec 32 Go de RAM peut héberger 5 VM configurées à 8 Go chacune (soit 40 Go virtuels alloués), si l'on estime qu'elles n'atteindront jamais toutes leur pic de consommation en même temps.

Risque

Si plusieurs VM réclament effectivement leur maximum simultanément, l'hyperviseur doit arbitrer — ralentissement généralisé, voire blocages. Le surengagement est une pratique courante en production, mais nécessite une **surveillance** attentive de la charge réelle.

10. Partie X — Migration de VM

Déplacer une VM d'une machine à une autre

Migration à froid

La VM est **arrêtée**, ses fichiers (disque virtuel, configuration) sont copiés vers une autre machine physique, puis la VM y est redémarrée — simple, mais implique une **interruption de service**.

Migration à chaud (*live migration*)

La VM est déplacée vers une autre machine physique **sans interruption perceptible** pour ses utilisateurs — technique utilisée en production (ex. VMware vMotion, migration en direct de Hyper-V) pour la maintenance de serveurs physiques sans coupure de service.

À retenir

VirtualBox (Type 2, usage pédagogique/personnel) ne propose pas de migration à chaud native — cette fonctionnalité avancée est typique des hyperviseurs de Type 1 utilisés en production, notamment dans les infrastructures Cloud du Chapitre 1.

11. Partie XI — Virtualisation vs conteneurisation

Un premier aperçu, avant le chapitre dédié

Critère	VM (ce chapitre)	Conteneur (chapitre suivant)
Système d'exploitation	Un OS complet par VM	Partage le noyau de l'hôte
Taille	Plusieurs Go	Quelques Mo à centaines de Mo
Démarrage	Minutes	Millisecondes à secondes
Isolation	Très forte (niveau matériel)	Forte (niveau processus)

À retenir

Tout ce qui a été appris dans ce chapitre (isolation, ressources virtuelles, snapshots, clonage) prépare à comprendre **par contraste** pourquoi la conteneurisation (Docker) a été inventée — et dans quels cas continuer à préférer une VM classique (isolation forte, OS différent de l'hôte, ex. exécuter Windows sur un hôte Linux).

12. Étude de cas de synthèse

Scénario intégrateur

Contexte

Un enseignant doit préparer un environnement de TP pour 25 étudiants, identique pour chacun, avec Ubuntu Server et quelques logiciels préinstallés. Le poste utilisé pour préparer cet environnement dispose de 32 Go de RAM et 8 curs CPU.

À déterminer :

1. Quel type d'hyperviseur utiliser, et pourquoi ?
2. Comment obtenir 25 environnements identiques sans réinstaller 25 fois manuellement ?
3. Quelle protection mettre en place avant de laisser les étudiants manipuler librement leur VM ?
4. Combien de VM à 2 Go de RAM chacune peut-on raisonnablement exécuter simultanément sur ce poste, en réservant de la RAM pour l'hôte ?

Solution commentée

1. **Type 2 (VirtualBox)** : usage pédagogique sur un poste personnel, pas de besoin de performance de production ni d'infrastructure datacenter.
2. Créer **une VM de référence**, l'installer et la configurer entièrement, puis en faire **25 clones complets**, chacun indépendant.
3. Créer un **snapshot** de chaque VM juste après le clonage, afin de pouvoir restaurer rapidement l'état initial si un étudiant endommage son environnement.
4. En réservant environ 4 à 6 Go pour l'hôte et le système, il reste environ 26-28 Go disponibles, soit une douzaine de VM à 2 Go **simultanément** sans surengagement excessif (les VM des étudiants ne tournant pas forcément toutes en même temps sur le même poste, un raisonnement d'overcommitment pourrait aussi s'appliquer si le poste est partagé).

13. Synthèse

Tableau de synthèse final

Concept	Savoir-faire
VM / Hyperviseur	Expliquer le rôle de chacun et leur relation
Type 1 vs Type 2	Choisir le type adapté à un contexte donné
Ressources virtuelles	Dimensionner RAM/CPU sans sur- ou sous-allouer
Disques virtuels	Choisir un format et un mode d'allocation adaptés
Snapshots	Utiliser à bon escient, sans les confondre avec une sauvegarde
Clonage	Choisir entre clone complet et clone lié
Overcommitment	Comprendre le compromis risque/économie
Migration	Distinguer migration à froid et à chaud

14. Mini-quiz

Mini-quiz (1/2)

1. Quelle est la différence entre un hyperviseur de Type 1 et de Type 2 ?
2. VirtualBox est-il un hyperviseur de Type 1 ou Type 2 ?
3. Qu'est-ce qu'un disque virtuel, physiquement, sur la machine hôte ?
4. Quelle est la différence entre un disque à taille fixe et un disque dynamiquement alloué ?
5. Qu'est-ce qu'un snapshot ?

Mini-quiz (2/2)

6. Un snapshot est-il une stratégie de sauvegarde fiable ? Justifier.
7. Quelle est la différence entre un clone complet et un clone lié ?
8. Qu'est-ce que l'*overcommitment* de ressources ?
9. Quelle est la différence entre une migration à froid et une migration à chaud ?
10. Une VM partage-t-elle le noyau de sa machine hôte ?

Corrigé du mini-quiz

1. Type 1 : installé directement sur le matériel (bare-metal). Type 2 : installé comme application par-dessus un OS hôte existant.
2. **Type 2** (hosted).
3. Un **fichier** sur le disque physique de l'hôte, présenté à la VM comme un disque dur complet.
4. Taille fixe : espace réservé intégralement dès la création, meilleures performances. Dynamique : croît progressivement selon l'usage réel, plus flexible.
5. Une capture de l'état complet d'une VM à un instant donné, permettant d'y revenir ultérieurement.
6. Non : il est stocké sur le même disque physique que la VM ; en cas de perte du disque, le snapshot est perdu aussi.
7. Clone complet : copie intégrale, totalement indépendante. Clone lié : ne stocke que les différences, dépend de la VM d'origine.
8. Allouer sur le papier plus de ressources virtuelles que la capacité physique réelle, en pariant qu'elles ne seront jamais toutes sollicitées au maximum simultanément.
9. Migration à froid : VM arrêtée puis déplacée (interruption). Migration à chaud : déplacement sans interruption perceptible.
10. **Non** : contrairement à un conteneur, une VM exécute son propre système d'exploitation complet, indépendant du noyau de l'hôte.

15. TP guidé

TP guidé (45–60 min)

1. Installer VirtualBox (si non déjà fait) et vérifier son bon fonctionnement
2. Créer une nouvelle VM (nom, type, version), avec 2 Go de RAM et 20 Go de disque dynamiquement alloué
3. Associer une image ISO Ubuntu Server et procéder à l'installation
4. Installer les Guest Additions et tester le partage de presse-papiers
5. Créer un dossier partagé entre l'hôte et la VM
6. Créer un snapshot juste après l'installation initiale
7. Modifier volontairement un fichier système, puis restaurer le snapshot pour observer le retour à l'état initial
8. Cloner la VM (clone complet) et vérifier son indépendance en démarrant les deux VM simultanément
9. Observer, dans le Gestionnaire VirtualBox, la consommation de RAM et CPU allouée à chaque VM active

16. Conclusion

Résumé du chapitre

- Une **VM** est un ordinateur complet émulé, géré par un **hyperviseur**, qui alloue et isole les ressources physiques réelles.
- Les hyperviseurs **Type 1** (production, Cloud) et **Type 2** (VirtualBox, pédagogie) répondent à des besoins différents.
- Un disque virtuel est un simple **fichier**, à taille fixe ou dynamique.
- Les **snapshots** et le **clonage** facilitent le développement, le test et la pédagogie — mais un snapshot n'est pas une sauvegarde.
- La gestion des ressources partagées (*overcommitment*) est un compromis central entre économie et fiabilité.

Prochaine étape : les réseaux virtuels, la sécurité et l'automatisation avec Vagrant.